

The Chakravayuh Gazette

AN ORGAN OF RECORD FOR THE DEFENCE OF VAYU-SUCHAK 2.0

VOL. I · NO. 1

PUBLISHED BY TEAM CHAKRAVYUH — SMART INDIA HACKATHON 2026

PRICE: YOUR COMPOSURE

Every Question the Judges Can Ask

*A complete field manual for defending the real-time airfare price index
— the pitch, the pipeline, the statistics, and the awkward questions —
written so a first-time team can answer all of them from principles.*

THE NEWSROOM / A STANDING BRIEF

VAYU-SUCHAK 2.0 is a desktop tool that automatically collects airfares from airline and travel-aggregator websites, cleans them with a transparent statistical filter, separates festival-driven spikes from real price change, and combines the result into an official-style price index — a single number, base 2022 = 100, that tracks how air-travel prices are moving across India so MoSPI can feed it into the Consumer Price Index.

The problem statement, SIH26056, asks for “a Real-time Airfare Price Index for India through Automated Web Scraping of Airline and Online Travel Aggregator Portals for Augmentation of CPI.” Every clause is a thing the judges can test: *real-time, web scraping, airline and OTA portals, index, augmentation of CPI*. This paper walks each one.

ONE RECORD'S JOURNEY

A fare is scraped, normalised, tagged if it departs within seven days of a festival, tested against a two-tailed IQR fence and either kept or logged as an anomaly, averaged with its route's other fares, weighted by that route's base-year passenger volume, folded into the Laspeyres sum, and written to PostgreSQL — or a SQLite fallback if Postgres is down. Every stage prints one line to a live console. Nothing is deleted silently; every number traces to a public source.

WHAT YOU WILL SHOW

The operator dashboard running the pipeline end to end, with two failures staged — a blocked scrape and a downed database — to prove the system degrades instead of collapsing. Still being wired: the seam between the dashboard and the live scrapers. The statistical core is built and unit-tested against hand-calculated numbers — say that plainly if asked.

Each desk below is one topic; each item is a question, a plain answer, and the line to say aloud. Don't memorise 110 answers — read it once so the team shares one model.

KNOW THESE COLD

PROBLEM STATEMENT

SIH26056 — MoSPI — Travel & Tourism — Software

PROJECT

VAYU-SUCHAK 2.0 (“Vayu” = air, “Suchak” = index)

INDEX

Weighted Laspeyres, base period 2022 = 100

FORMULA

$$\text{Index} = \left[\frac{\sum (P_t \cdot Q_0)}{\sum (P_0 \cdot Q_0)} \right] \times 100$$

PT / P0 / Q0

current avg price / base-year price / base-year passenger volume — per route

DEMO CORRIDORS

DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, DEL-GAU (GAU proves weighting)

WORKED NUMBER

118.75 — naive average would wrongly say 132.5

OUTLIER FILTER

Two-tailed IQR, fence $[Q1 - k \cdot \text{IQR}, Q3 + k \cdot \text{IQR}]$, default $k = 1.5$

FESTIVAL WINDOW

± 7 days of a gazetted festival, dates from the holidays library

SCRAPE SOURCES

MakeMyTrip, Ixigo, Cleartrip, IndiGo, Air India

BACKEND

Python 3.11, FastAPI + uvicorn (async), pandas, httpx, Playwright, SQLAlchemy Core

DATABASES

PostgreSQL primary → SQLite WAL fallback

LIVE CONSOLE

Server-Sent Events, one GET endpoint, browser EventSource

FRONTEND

one index.html, vanilla JS + Chart.js, pywebview shell

PROVENANCE

P0 ← Kaggle 2022 fares · Q0 ← DGCA airport-pair volumes

COST

₹0 — every tool free / open source

FIVE THINGS TO FIX FIRST

1. **Positioning.** The deck leads with “dual-mode: API + scraper fallback.” The engineering doc reversed that — scraping is primary — and the PS says “through Automated Web Scraping.” Realign the deck to lead with scraping.
2. **IQR slide typo.** Slide 4 reads Upper Fence = $Q2 + 1.5 \times IQR$. Should be $Q3$, and k not a hard 1.5 — the knob is the whole point.
3. **Team ID** on the title slide is still [Portal Team ID]. Fill it in.
4. **The dashboard index is simulated** ($\text{base} + (k - 1.5) \times 0.42 + \text{festival}$) and the CPI line is a synthetic drift. Know this. If a judge does the arithmetic, say the front-end is a faithful simulation and the real Laspeyres / IQR math is unit-tested separately.
5. **The AI layer is on no slide.** Self-healing extractor, nowcasting, anomaly model — add one line so “where’s the AI?” has somewhere to point.

Already done for you: the SIH26057 typo in the engineering deep-dive and the dashboard is corrected to SIH26056 to match the deck.

THE 40-SECOND OPENING

“Airfare is one of the most volatile prices in the economy, and government statistics still track it by hand. VAYU-SUCHAK automates that: it scrapes fares from airline and OTA portals the way the problem statement asks, cleans them with a tunable statistical filter where every exclusion is logged, separates festival spikes from real inflation, and weights each route by how many people actually fly it — the same method CPI itself uses. The result is a reproducible airfare index, base 2022, that MoSPI can feed straight into CPI. You’re about to watch it run live, including what happens when a site blocks us and when the database goes down.”

SECTION A

The Newsroom Briefing

FRAMING & PITCH — 8

A1 In one sentence, what is VAYU-SUCHAK?

A desktop tool that automatically collects airfares from airline and travel websites, cleans them, and turns them into an official-style price index — a single number that tracks how air-travel prices move across India, so MoSPI can feed it into CPI.

SAY

It replaces manual airfare-spreadsheet auditing with a transparent, reproducible pipeline you can watch run live.

A2 What does “augmentation of CPI” mean?

CPI measures inflation across a basket of goods and services. Airfares move fast and are collected slowly and by hand, so CPI under-captures them. We produce a defensible airfare sub-index that MoSPI can plug in to make that part of CPI more current and accurate.

SAY

We’re not replacing CPI — we’re building one high-quality input to it.

A3 Who is the end user?

A statistical officer at MoSPI. One operator, one machine. They pick a corridor and a date range, set the sensitivity knob, press one button, and read the index.

A4 Why is this a real problem, not a toy?

Airfare changes by route, by how far ahead you book, by demand, by time of day. Statistics teams track it manually today — slow, inconsistent, hard to audit. A national statistics office needs a method that is automated, repeatable and explainable.

A5 What makes your approach different from “scrape and average”?

Four things: we keep the context of every fare (route, timestamp, lead-time); a tunable filter removes data-error prices with every exclusion logged; a festival flag separates holiday spikes from real inflation; and volume weighting makes busy routes count more — the same method CPI uses.

A6 Is this built, or a concept?

The dashboard is built and runnable — the operator experience, the live console, the charts, the failover demo. The backend pipeline is specified stage-by-stage with unit tests for the math; integration is the remaining work.

ADJUST TO YOUR TRUE STATUS ON THE DAY. Never claim more than runs.

A7 What does the name mean?

“Vayu” is air, “Suchak” is an index or indicator — literally “air-fare index,” fitting for a government-statistics product.

A8 Why should we pick your team?

We treated this as a statistics problem, not a scraping stunt. Every number traces to a public source, every exclusion is logged not deleted, every design choice matches how real price indices are built, and the whole thing runs offline for ₹0. It is defensible line by line.

SECTION B

The Problem-Statement Desk

PS FIT — 8

B1 The PS says “web scraping.” Do you scrape?

Yes — that is the primary acquisition method, exactly as the PS specifies. We drive real headless browsers against MakeMyTrip, Ixigo, Cleartrip, IndiGo and Air India, and intercept each site’s own fare data.

SAY

The PS asks for automated web scraping of airline and OTA portals; that’s what we do. The volume weights come from official DGCA data — those we don’t scrape.

B2 Isn’t scraping these sites against their terms?

The PS explicitly directs us to scrape these portals for a national-statistics purpose — public interest, not resale. We scrape responsibly: honour robots.txt, one request every few seconds per site, off-peak, realistic user-agent, and cache everything so we don’t re-hit them. In production MoSPI would formalise this with data-sharing agreements; the prototype demonstrates the method.

B3 The PS says “standalone desktop application.” Is a browser tab that?

No — which is why we wrap the whole thing in `pywebview`, about fifteen lines that give it a native OS window with its own title and icon. It launches with `python run.py` as a real application, no browser visible.

B4 What does “real-time” mean for a monthly inflation input?

The scrapers run on a schedule and continuously write a cache; the index always reads the freshest fares. On stage we scrape one source live to prove the path and serve the rest from cache that is minutes-to-hours old — well inside tolerance for a monthly CPI input.

B5 Why is this under Travel & Tourism?

Because the raw material is airline and OTA fare data. But the output is an economic-statistics product — the theme is where the data lives, not what the deliverable is.

B6 Your deck says API-first; the PS says scraping. Which is it?

Scraping is primary, per the PS. An early version of our plan over-weighted APIs because free flight APIs are unreliable; we corrected that. The scraper pool is the acquisition layer; a cached snapshot is the fallback when a site blocks us mid-run.

FIX THE SLIDE BEFORE THEY ASK. See front page, item 2.

B7 Does your solution address every verb in the PS?

Yes — one pipeline stage per verb: scrape (collect) → normalise (split base fare / tax / total) → festival flag → IQR purifier (detect anomalies) → weighted Laspeyres (weight) → persistence + dashboard + history chart (publish a reproducible series).

B8 What are you deliberately not doing?

No price forecasting — this measures history, it doesn’t predict. No reckless scraping. No multi-user accounts, auth, or cloud deployment — single-operator local app. No microservices — one process, one port.

C1 What is a price index, in plain terms?

A single number that compares prices now to prices in a fixed reference period. The reference is set to 100. If the index is 118.75, prices are 18.75% above that reference, on a volume-weighted basis.

C2 What does 118.75 actually mean?

Volume-weighted air fares across the measured routes are 18.75% higher than they were in the 2022 base year.

C3 What is the Laspeyres formula and why that one?

$\text{Index} = [\Sigma(P_t \cdot Q_0) / \Sigma(P_0 \cdot Q_0)] \times 100$. It compares what today's fares would cost at base-year passenger volumes against what base-year fares cost at those same volumes. Weights are frozen at the base year, so movements reflect price change only, not people changing their travel habits. That is exactly how CPI is built — which makes our number directly comparable to CPI.

C4 What are P_t , P_0 , Q_0 ?

P_t = current average price on a route, after cleaning. P_0 = that route's average price in the base year. Q_0 = that route's passenger volume in the base year, used as a fixed weight.

C5 Why volume-weighting, not a simple average?

A simple average treats Delhi–Mumbai the same as a tiny regional route. If a small route's fare jumps, a plain average lets it swing the national figure. Weighting by passenger volume makes the route a thousand people fly matter ten times more than the route a hundred fly.

C6 Show me the weighting effect with numbers.

DEL–BOM: P_t 5750, P_0 5000, Q_0 1000. DEL–GAU: P_t 9000, P_0 6000, Q_0 100. Laspeyres = $(5750 \cdot 1000 + 9000 \cdot 100) / (5000 \cdot 1000 + 6000 \cdot 100) \times 100 = 6,650,000 / 5,600,000 \times 100 = 118.75$. Naive average of the two price ratios = $(1.15 + 1.50) / 2 = 132.5$ — 14 points too high, because the tiny route dominates.

C7 Why Laspeyres, not Paasche, Fisher, or a chained index?

Paasche needs current-period volumes every month — data we don't have in real time. Fisher is the geometric mean of both, so it inherits that problem. A chained index reweights every period. Laspeyres with fixed base weights is simplest, needs the least data, and matches CPI's own method.

C8 Why 2022 as the base year?

Recent enough to be relevant, after the COVID fare distortions of 2020–21, and there is a public Kaggle dataset of Indian domestic fares from that period we can derive P_0 from. It is a config value, not hardcoded logic — MoSPI could rebase it.

C9 Isn't 2022 still a strange year for air travel?

It is a defensible, explicit choice. If MoSPI prefers a different base, the reference file swaps out and the whole series recomputes. The method doesn't depend on which year we pick.

C10 How is one run an "index"? An index is a time series.

Correct — one run is one data point. We run the pipeline over N historical months and plot index-vs-month, overlaid with real CPI. The dashboard's history chart is that series building up run by run.

C11 How do you handle a route with no fares in a period?

Laspeyres only includes routes present in both the base file and the current data — an inner join. Optionally the nowcasting model imputes a missing route-month from correlated routes and history, with a confidence band; the index is valid without it.

C12 What's the relationship to the actual CPI transport component?

Ours is a sub-index — one input. MoSPI's CPI transport group covers much more than airfare. We feed our airfare index in as the airfare line item, at higher frequency and with a documented method.

C13 The dashboard's CPI comparison line — where's that from?

In the current build that line is a synthetic reference drift for visual context — we don't scrape CPI. The real version plots MoSPI's published CPI transport monthly series as a static array. If we can't source it before the deadline we remove the line rather than ship a fake one.

DO NOT CLAIM IT IS REAL. Say "context drift" or drop it.

C14 What if base-year prices are wrong?

P_0 comes from a public dataset and is frozen in a file anyone can inspect. If it is revised, we rebase and note it — same as any statistical office. The point is it is external and cited, not invented.

D1 Where does Po, the base price, come from?

A public Kaggle dataset of roughly 300,000 historical Indian domestic flight fares. We script it to compute the mean 2022 fare per route.

D2 Where does Qo, passenger volume, come from?

DGCA's published airport-pair domestic passenger traffic, via data.gov.in. It is an official government statistic — we don't scrape it.

D3 What is DGCA?

The Directorate General of Civil Aviation — India's civil aviation regulator. It publishes official domestic traffic data.

D4 What if airport-pair DGCA data isn't available?

We fall back to passengers-per-airport as a proxy weight and say so explicitly in the methodology. It is a documented approximation, not a hidden one.

D5 Is the data in your demo real?

The base-year reference (P0, Q0) is real, derived from the sources above. The cache holds real previously-scraped fare snapshots. The dashboard's synthetic fare distribution is a stand-in used during frontend development; the real pipeline uses scraped fares.

D6 How many corridors, and why these five?

DEL-BOM, DEL-BLR, BOM-BLR, DEL-CCU, DEL-GAU. Four are among the busiest domestic routes; DEL-GAU is deliberately small so we can show that volume weighting changes the answer. The method scales to any number.

D7 How many fare observations per route?

We aim for a distribution of roughly 40 to 100 fares per route per collection, across dates and sources — enough for the IQR filter to be statistically meaningful.

D8 What's in base_year_reference.json?

A small frozen file: `{ "base_period": "2022", "routes": { "DEL-BOM": { "P0": 5100, "Q0": 1000 }, ... } }`. Frozen means reproducible — the same file always gives the same base.

D9 What about international routes?

Out of scope for the prototype — the PS and CPI context are domestic. The method is identical if you add international routes with their own P0 / Q0.

D10 Could airlines just give you this data directly?

In production, yes — via a MoSPI data-sharing MoU. The PS asks us to demonstrate the scraping-based acquisition method because that MoU doesn't exist yet.

E1 How does the scraper actually work?

One lightweight adapter per source. Each opens the fare-search page in a real headless Chromium via Playwright, waits for results to load, and grabs the fares — normally by reading the site's own background data response, and if that changes shape, by having a vision-language model read the rendered page.

E2 What is Playwright?

A free browser-automation library that drives a real Chromium browser programmatically — clicking, waiting, reading the page. Because it is a real browser running real JavaScript, it passes most bot checks that a bare HTTP request fails.

E3 Why a full browser instead of a plain HTTP client?

These sites are single-page apps — the fare list is drawn by JavaScript from the site's own internal API, using tokens and cookies that rotate. A bare client can't run that JavaScript and breaks silently. A real browser just works.

E4 What is XHR interception?

While the page loads it makes background requests to its own server for the fare list as structured JSON. We listen for that response and read the JSON directly — clean structured data, no HTML parsing.

SAY

We don't scrape HTML strings — we read the site's own JSON response, which is far cleaner.

E5 What's the "self-healing" part?

When a site changes its layout or its JSON shape, a normal scraper silently breaks. Ours detects the parse failure, takes a screenshot plus the page text, sends it to a vision-language model, and gets structured records back. It re-derives the fields itself — no overnight selector maintenance.

E6 Which LLM, and does it cost money?

A free tier — Groq or Google Gemini — or fully local via Ollama (`llama3.2-vision`). It is only called when the fast path fails, so usage is tiny.

E7 Isn't an LLM in a data pipeline a reliability risk?

It is the fallback, not the main path — primary extraction is deterministic JSON reading. Its output is validated against a strict schema before entering the pipeline. Garbage is dropped and logged, like any bad record.

E8 What happens when a site blocks you?

That adapter returns the newest cached scrape for that route and logs `[SCRAPE][CACHE] blocked → cached snapshot`. The other four sources still contribute. One source down never stops the run.

E9 Isn't running the demo on cache cheating?

No — everything in the cache is a real scrape we ran earlier, timestamped. "We scraped it earlier" is exactly what the PS asks for. On stage we scrape one source live; the rest come from a cache hours old at most.

E10 Then how is it real-time?

The scrapers run continuously on a schedule and keep the cache fresh; the index always reads the latest. For a monthly CPI input, fares hours old are current.

E11 Why not scrape all five sources live in the demo?

Slow (5 to 15 seconds per search per site), flaky, and one CAPTCHA on stage kills it. We pre-scrape four the night before and live-scrape the lightest one. If that is blocked, it drops to cache — and we demo that as a feature.

E12 How do you avoid getting IP-banned in production?

Rate limiting (one request per few seconds per domain), off-peak scheduling, rotating realistic user-agents, honouring `Retry-After`, and hard caching so we never re-fetch what we have.

E13 How do you dedupe fares across five sources?

Key on airline + flight number + departure timestamp. The same flight quoted on two sites collapses to one record; we keep the fares and note the sources.

E14 What fields does a fare record carry?

`flight_id`, `route`, `price`, `currency`, `departure_ts`, `airline`, `source` — the contract every downstream stage depends on — plus a base fare / tax / total split during normalisation.

SECTION F

The Statistics Desk

IQR & K_FACTOR — 12

F1 What is the IQR filter, in plain words?

It throws out prices that are clearly data errors — impossibly low (₹1, a null read as 0) or absurdly high (a glitch fare) — before they poison the average, using a standard statistics rule.

F2 What are Q1, Q3, IQR?

Sort the fares. Q1 is the 25th-percentile price, Q3 the 75th. $IQR = Q3 - Q1$, the spread of the middle half. Anything far outside that middle band is suspect.

F3 What are the fences?

Lower fence = $Q1 - k \cdot IQR$. Upper fence = $Q3 + k \cdot IQR$. Fares outside `[lower, upper]` are excluded. This is Tukey's rule — textbook.

F4 Why IQR and not standard deviation / z-score?

Airfares are right-skewed — a long tail of expensive last-minute and business fares. Mean and standard deviation get dragged by that tail, so a z-score rule would wrongly throw away legitimate high fares. Quartiles are robust to skew. That is why IQR is the standard tool for skewed data.

F5 What is k_factor?

The sensitivity knob. Bigger k = wider fences = fewer exclusions. Smaller k = stricter. Default 1.5, Tukey's classic value. Exposed in the UI, not hidden in code, so an analyst can tune and audit it.

F6 Prove k_factor actually does something.

Prices [4200, 4500, 4800, 5000, 5200, 5500, 5800, 6000, 9000, 15000]. Q1 = 4850, Q3 = 5950, IQR = 1100. At k=1.5, upper fence = 7600 → excludes 9000 and 15000 (2). At k=3.0, upper fence = 9250 → excludes only 15000 (1). Drag the slider on stage; watch the count change.

F7 Why two-tailed? Why a lower fence at all?

The PS itself mentions placeholder and glitch prices. Fare-error data shows up as impossibly low values — ₹1, nulls as 0. Screen only the top and those pass through and drag the index down. The lower fence catches them — three extra lines.

F8 What if an analyst sets k wrong?

That is the point of exposing it — so they can audit it. Every excluded fare is logged and stored with the reason and the fence value. Nothing is deleted. Any k choice is fully reversible and reviewable.

F9 Do you ever delete data?

Never silently. Anomalies are flagged, logged, and kept in storage with an exclusion reason. The index is computed on the clean set; the excluded rows stay on the record.

F10 Isn't a hard fence arbitrary at the boundary?

It is a transparent rule with one tunable parameter, which beats a black box. For borderline cases the anomaly log lets a human review them, and the ML anomaly model catches suspicious fares the fence passes.

F11 What's the anomaly count on a normal run?

Usually a handful out of a few hundred fares. If it is suddenly large, that is a signal the scrape itself went wrong — which the console surfaces.

F12 Your slide shows “Q2 + 1.5·IQR” — is that right?

It should read Q3, and k rather than a fixed 1.5 — the parameter is configurable. The code is correct; the slide has a typo we're fixing.

SECTION G

The Calendar Desk

FESTIVAL ENGINE — 7

G1 What does the festival engine do?

It tags every fare that departs within ± 7 days of a major Indian festival. That lets us compute the index two ways — with and without festival-season fares — and see how much of a move is just holiday demand.

G2 Why ± 7 days, not the exact date?

Airfare demand ramps up before a festival and stays high after — people travel home in the days around it. ± 7 days captures that window. It is a config parameter, not a magic number.

G3 Where do the festival dates come from?

The `holidays` Python library — `holidays.India(years=2026)`. Correct named dates, works offline, maintained upstream so we don't hand-keep a calendar.

G4 Why not hardcode the dates?

Diwali, Holi, Dussehra, Eid are lunar — their Gregorian dates shift every year. Hardcoding from memory means a wrong date a judge can catch live. The library is the safe choice.

G5 Which festivals?

Diwali, Holi, Dussehra, Christmas, Guru Nanak Jayanti, Eid al-Fitr, Eid al-Adha — the ones with real nationwide travel impact.

G6 What is the “ex-festival” index?

The same Laspeyres calculation with festival-flagged fares removed. The gap between the full index and the ex-festival index is the transient holiday-demand component — exactly what a statistical office wants separated from structural price change.

G7 Isn't festival flagging just cosmetic?

No — it is a decomposition. `index_all` vs `index_ex_festival` tells MoSPI how much of this month's airfare move is a temporary Diwali spike versus a real trend. A genuine analytical output.

H1 Where is the AI in this?

Four places, all optional to the core index: the self-healing vision-LLM extractor in the scraper; an anomaly-detection model that flags decoy or placeholder fares the IQR fence misses; a nowcasting model that fills gaps in the data; and an explainability model that attributes each index move to a likely cause.

H2 What does the anomaly model add over IQR?

IQR catches values that are numerically extreme. A learned model (Isolation Forest) can flag fares that look normal in price but are suspicious in pattern — decoy fares, bot-trap prices, placeholder responses. Both run; both are logged; nothing is silently removed.

H3 Nowcasting vs forecasting?

Forecasting predicts future prices. Nowcasting fills in missing present observations — if a route-day scrape failed, we estimate that fare from correlated routes and recent history, with a confidence band. It is how central banks nowcast GDP before the official figure lands.

SAY

We measure, we don't predict. Nowcasting is imputation of gaps, not a price forecast.

H4 Why not use ML for the whole index?

Because this is a measurement mandate, not a prediction problem. A price index has to be reproducible and explainable — a statistical office defends every number to the public. ML adds model risk and a black box with no measurement benefit here.

H5 So why use ML at all?

Only where it genuinely helps and stays auditable: making the scraper robust to layout changes, catching decoy fares a rule misses, and filling data gaps with stated uncertainty. The core index stays transparent arithmetic.

H6 Is the AI required for the demo?

No. The index is valid and defensible with just scrape → festival → IQR → Laspeyres. The ML pieces are “if time permits” enhancements. We'll be clear about what's live.

H7 What is Isolation Forest?

An unsupervised anomaly-detection algorithm from scikit-learn. It isolates unusual points by how few cuts it takes to separate them from the rest — outliers isolate fast. Free, standard, fast.

H8 What does the explainability model do?

An LLM reads airline press releases and news RSS around an index move and writes a one-line likely cause: “Index +3.2% — Diwali window plus IndiGo fuel-surcharge revision announced Oct 14.” It makes the index explainable, not just a number.

H9 How do you stop the LLM hallucinating a cause?

It is annotation, not computation — it never changes the index. The cause line is clearly labelled as model-generated with its sources linked, so a human can check it.

H10 Your mentor wanted an AI angle — is this bolted on?

The one genuinely AI-native piece is the self-healing extractor, and it solves a real problem the PS names: brittle scrapers that break on every UI change. The rest are targeted, optional, and kept out of the measurement path on purpose.

I1 What is FastAPI and why did you pick it?

A modern Python web framework. We picked it because it is async — the live log stream and the fare-fetching run concurrently inside one request without blocking each other. It also validates incoming requests automatically.

I2 What does async buy you here?

While the pipeline waits on a slow scrape, the event loop is free to keep pushing console lines to the browser. A synchronous framework would freeze the UI during the fetch.

I3 What is SSE?

Server-Sent Events — a one-way channel from server to browser over plain HTTP. The server keeps the connection open and pushes text lines; the browser's built-in `EventSource` receives them. It is how the console streams each pipeline stage as it happens.

I4 Why SSE and not WebSockets?

WebSockets are bidirectional and heavier to set up. Our console only needs server → browser. SSE is simpler, built into the browser, needs no extra library, and maps cleanly onto FastAPI's async generators.

I5 What is the orchestrator?

One file — an async generator `run_audit()` that runs each pipeline stage and yields one log line after it, with a short pause so the console visibly streams. It is the only file that knows about streaming.

I6 Why are the pipeline functions “pure”?

Each takes plain data in and returns plain data out — no web server, no database, no logging inside. You can import `pipeline.iqr` in a shell and test it with a list of numbers. It let three people build five modules in parallel and makes each easy to explain and test alone.

I7 One process or many?

One. One FastAPI process on one port serves both the static frontend and the API. No microservices, no port conflicts, nothing to orchestrate.

I8 What is Pydantic validation / the 422?

Request parameters are checked against a typed schema at the API boundary. Bad input — malformed corridor, end date before start, k out of range — returns a clear 422 before any pipeline code runs, instead of a cryptic error three modules deep.

I9 What is uvicorn?

The server process that runs a FastAPI app — it speaks the async server standard (ASGI) that FastAPI is built on.

I10 How does the frontend trigger a run?

It builds a query string and opens `new EventSource("/api/execute-audit?...")`. Each streamed line appends to the console. The final message carries `done:true` plus the index values, which triggers the big number, the chart refresh, and re-enables the button.

I11 Why GET for something that does work?

The browser's `EventSource` only does GET. Parameters ride in the query string, the response is the event stream — no separate “start job” call, no job registry, nothing to leak.

I12 What are the API endpoints?

`GET /` (dashboard), `GET /api/corridors` (dropdown options), `GET /api/execute-audit` (run and stream logs), `GET /api/runs` (past runs for the history chart).

J1 Why two databases?

PostgreSQL is the authoritative store — production-grade, handles concurrent writes. SQLite is a local-file fallback so a database outage never freezes the app mid-demo. Postgres is the store; SQLite is the safety net.

J2 What is PostgreSQL?

A full open-source relational database that runs as a server. We run it in Docker with one command, or use a free Neon cloud instance.

J3 What is SQLite?

A database that is just a single file on disk, no server. Always available, can't fail to “connect.”

J4 What is WAL?

Write-Ahead Logging — SQLite writes changes to a side `-wal` file first, then folds them into the main database. A reader and a writer don't block each other, and a crash mid-write can't corrupt the file. The row is durable the instant we commit.

J5 How does the failover work?

`persist_run()` tries Postgres with a 3-second timeout. On an `OperationalError` it catches the exception, connects to `db/fallback.sqlite`, sets WAL mode, inserts the same row, and returns `"sqlite_fallback"`. The row is tagged with which store holds it. Demo: `docker stop Postgres`, click Execute, watch `[DB][FALLBACK]`.

J6 What if BOTH databases fail?

SQLite is a local file, so a true dual failure means the disk is gone. We'd need an in-memory retry queue that flushes when a database comes back. That is stated future work — we don't claim it is solved.

ANSWER THIS ONE HONESTLY. Do not oversell.

J7 Isn't falling back to SQLite a downgrade?

It is a resilience net, not a replacement. Postgres stays authoritative. The row is tagged `db_target='sqlite_fall-back'` so you always know which store holds it and can reconcile later.

J8 Why not just use SQLite everywhere?

For a national system, concurrent writes and proper operational tooling matter — that is Postgres. SQLite is there for when Postgres is temporarily unreachable.

J9 What's stored per run?

`run_id`, corridor, date range, index value, ex-festival index, `k_factor` used, base period, records ingested, festival-flagged count, anomalies-excluded count, scrape sources, `db_target`, timestamp. Same schema in both databases.

J10 How does this prove reproducibility?

Pure functions + frozen reference files + a stored run record for every execution. `run_id` is a hash of (corridor + dates + `k_factor` + data-snapshot date), and the SHA is a hash of the same string. Same inputs → same run id → same index, every time. The seal on the dashboard is that hash.

SECTION K

The Control Room

THE DASHBOARD — 10

K1 What's actually built in the frontend?

The full operator cockpit: corridor / date / `k_factor` controls, the live streaming console with stage tags and latency stamps, a five-stage pipeline visual, a national map with corridor arcs, the split-flap index readout, an IQR histogram, an index-history chart, an anomalies table, a scrape-source health monitor, a degraded / fallback banner, a methodology drawer, a corridor-comparison drawer, and a run-replay scrubber.

K2 Is it connected to the backend?

Not yet — it runs a faithful simulation of the pipeline's behaviour and timing. Backend integration swaps the simulation for a real `EventSource` stream with no change to the visuals. The simulation stays as an offline demo mode.

K3 Is the index number on screen real math?

In the current build it is a simulated value that responds correctly to the `k_factor` and the festival window. The real Laspeyres and IQR math live in the pipeline modules with unit tests against hand-calculated numbers.

BE FIRST TO SAY "SIMULATED." Don't get caught doing the arithmetic.

K4 Why the "dormant → live" boot animation?

Every animation marks a real event — data arriving, a stage completing, a value changing. It is the UI executing a real pipeline step, not decoration. Before Execute the screen is calm; pressing it powers the instrument on, stage by stage.

K5 What is progressive disclosure?

Panels only appear when they have something real to say. The anomalies table stays hidden at zero. The history chart says "first observation" on run 1. Fallback badges appear only if a fallback fired. Nothing shows fake state.

K6 What's the split-flap number?

The airport-departure-board-style index readout. Each digit flips only when its value changes, cascading left to right — so a re-run where only the last digit moves shows exactly that.

K7 What does the national map show?

The five corridors as arcs between city dots. Each arc's colour interpolates green → amber → red by that route's sub-index. The selected corridor is thicker and glowing, with a travelling dash during a run.

K8 Why vanilla JS and Chart.js — no framework?

No build step, no npm, a beginner can read every line, and it runs smoothly inside `pywebview` on a modest laptop. The charts and map are hand-rolled SVG.

K9 What's the reproducibility seal?

A stamp that rotates in on completion: `RUN <id> · SHA <hash> · REPRODUCIBLE`. Same inputs produce the same seal — the visible proof of determinism.

K10 Does it work on the venue projector?

Built to fit 1366×768 and 1280×720 with the anomalies table visible without scrolling, fonts embedded so it works off-line, and a reduced-motion mode that keeps every state change but drops the animation.

SECTION L

The Live Broadcast

THE DEMO ITSELF — 9

L1 Walk us through your demo.

Open on the dormant screen. Pick DEL-BOM, a Diwali-window date range, $k=1.5$. Press Execute. The console streams each stage — scrape, festival, IQR, Laspeyres, DB commit. The index split-flaps to its value, the map recolours, the history chart draws. Then the failure drills.

L2 “What if the API / scrape is down?”

Turn off wifi, click Execute. Console: [SCRAPE][FALLBACK] ... → cached snapshot. The run completes on cache. Shown as designed resilience.

L3 “What if the database dies?”

`docker stop Postgres`, click Execute. Console: [DB][FALLBACK] Postgres unreachable → SQLite WAL commit OK. Row still persisted.

L4 “Show me k _factor is real.”

Drag the slider $1.5 \rightarrow 3.0$, re-run. Console: [IQR] $k=3.0$... 1 excluded (was 2). The histogram fence sweeps, one red bar turns blue, the anomaly count drops.

L5 “Prove it’s reproducible.”

Run the exact same inputs twice. Identical Index = line, identical run id and SHA in the seal, both times.

L6 What if the demo crashes on stage?

The offline simulation mode is always available and needs no network or database. If the live path fails, we switch to it and keep talking — and having a clean fallback is itself part of the story.

L7 How long is the full boot sequence?

About five seconds for a first run. A re-run — just changing k — is a fast ~800ms recompute, no full reboot.

L8 What does the console prove?

Every stage tag appears in order — [SYSTEM] [SCRAPE] [FESTIVAL] [IQR] [LASPEYRES] [DB] — with a running latency timestamp. A live audit trail of the pipeline, not a loading spinner.

L9 Can we touch it?

Yes — that is the point. Every control is exposed. The worst a judge can do is change a corridor or a k value, and the UI stays consistent because the readout, the map, and the ticker all read the same number.

SECTION M

The Business Desk

FEASIBILITY, COST, SCALE, TEAM — 10

M1 How is the cost ₹o?

Every tool is free and open source: Python, FastAPI, Playwright, pandas, scikit-learn, PostgreSQL / SQLite, Chart.js. The LLM fallback uses free API tiers or a local model. The data sources are public. No paid APIs, no cloud bill.

M2 How long to finish?

A staged build — scaffold, frontend shell, the SSE bridge (hardest, de-risked first), then each pipeline module with its unit test, then the scraper pool, then persistence, then polish. Roughly a five-day skeleton for a small team, compressible or stretchable.

M3 Team size and split?

Team Chakravyuh. The modules are independent pure functions, so they are built in parallel — one owner per module (frontend, scraper, festival, IQR, Laspeyres, DB, integration). Only the integration seam needs everyone’s code to exist.

M4 Can this run at national scale?

The method scales linearly — add routes with their P0 / Q0, add source adapters, run the scraper pool concurrently. The bottleneck is polite scraping rate limits, which is a scheduling problem, not an architecture one.

M5 What's the production deployment path?

Scraper pool as scheduled workers → shared Postgres → the FastAPI index service and dashboard on an internal MoSPI server. Formalise site access with data-sharing MoUs. Everything we built maps onto that without a rewrite.

M6 What would MoSPI have to do to adopt this?

Decide the route basket and base year, sign data-access agreements with the airlines / OTAs (or take the scraping route officially), and run it on their infrastructure. The methodology and code are the deliverable.

M7 What's the hardest part technically?

Keeping the scrapers alive as five sites independently change their layouts. That is exactly why the extraction layer is a vision-LLM that self-heals instead of hardcoded selectors.

M8 What's your biggest project risk?

Live scraping reliability on demo day. Mitigated by pre-scraping into cache and live-scraping only one lightweight source, with a rehearsed fallback.

M9 Why not buy fare data from a provider?

Commercial fare data is expensive and licence-restricted, and the PS specifically asks us to demonstrate the automated collection method. Our approach is ₹0 and reproducible by anyone.

M10 What happens when free API limits run out in testing?

An on-disk JSON cache keyed by corridor + dates. Practice runs hit the cache — instant and safe. We show one real cache-miss → live call.

SECTION N

The Editorial Page

IMPACT & "SO WHAT" — 7

N1 Who benefits, and how?

MoSPI gets a faster, documented airfare input to CPI. Better CPI means better-informed monetary policy, dearness-allowance calculations, and contract indexation — all of which move real money for millions of people.

N2 What decision does this actually inform?

The airfare component of headline inflation. If air travel is inflating faster than CPI currently reflects, policymakers are working with a stale number.

N3 Why is a government index better than a private fare tracker?

Private trackers optimise for booking conversion, not statistical rigour. A CPI input has to be method-transparent, reproducible, weighted by real consumption, and auditable — which is what we built for.

N4 Daily, weekly, or monthly?

The pipeline can run at any frequency. CPI is monthly, so the headline output is monthly, but MoSPI can pull weekly views for early signal.

N5 What are the honest limitations?

It measures the routes in the basket, not every route. It depends on scrape coverage. The festival window is a heuristic. The base year is a choice. All documented, none hidden.

N6 What's genuinely novel versus existing work?

The combination: a self-healing scraper feeding a transparent, CPI-method-matched index with festival decomposition and full provenance, packaged as a one-operator desktop tool that runs offline. Each piece exists somewhere; the assembled, defensible whole is the contribution.

N7 If we gave you two more months?

Full historical backfill for a multi-year published series, the nowcasting model with proper confidence bands, more routes, and a formal validation report comparing our series to MoSPI's existing transport CPI.

01 Isn't this just a calculator with a fancy UI?

The calculation is deliberately simple — an index has to be. The work is everything around it: reliable acquisition from hostile sources, transparent cleaning, correct weighting, festival decomposition, provenance, resilience, reproducibility. A calculator doesn't survive a site changing its layout or a database going down.

02 You made these numbers up.

Every input traces to a public source — P0 from a Kaggle historical dataset, Q0 from DGCA via data.gov.in, festival dates from a maintained library, fares from the scrapers. The demo's synthetic distribution is a development stand-in; the real pipeline uses scraped fares, and the math is unit-tested against hand calculations.

03 The number on screen contradicts the map / ticker.

They always agree, because the readout, map arc, and ticker all read the same sub-index value for the selected corridor after each run.

ONLY TRUE ONCE THE F8.1 FIX IS IN. Verify before you present.

04 What's the weakest part — honestly?

Right now, the seam between the polished frontend and the pipeline. The dashboard simulates; the modules compute. Wiring them end-to-end with a live scrape is the remaining integration work, and the highest-risk piece.

05 What breaks first at scale?

Scrape freshness. More routes and more sources means more scheduling and more sites that can block you. It is an operational load problem — the index math doesn't change.

06 Why should MoSPI trust a scraper over a survey?

They shouldn't trust it blindly — they should trust the method, which is why every exclusion is logged, every source cited, every run reproducible from its inputs. A transparent, auditable scraper beats a slow, inconsistent manual survey.

07 What if an airline sues over the scraping?

The prototype demonstrates the method the PS asks for, for a public-statistics purpose, with responsible-scraping controls built in. Production would run on data-sharing agreements — the same way MoSPI already works with other data providers.

08 Your IQR filter could remove a real price surge.

It removes individual fares that are statistical errors, not a broad market move. If every fare on a route rises, Q1 and Q3 rise with them and nothing gets excluded — the index correctly goes up. The fence only catches values detached from the distribution.

09 This depends on free tiers and datasets that could vanish.

The datasets are downloaded and frozen in the repo, not fetched live. The LLM fallback runs locally if the free tiers disappear. The only live dependency is the airline / OTA sites themselves — inherent to the problem, and cached against.

010 What did you learn building this?

That the hard part of a statistics product isn't the formula — it's making every step defensible. We rewrote our own plan once when we realised "we use an API" wasn't answering the problem the PS actually set.

IF THEY SAY ... YOU SAY

TERM DROPPED	THE LINE
CPI augmentation	<i>CPI under-captures fast-moving airfares; we produce a defensible airfare sub-index to feed in.</i>
Index base 100	<i>118.75 means fares are 18.75% above the 2022 base, volume-weighted.</i>
Laspeyres	<i>Price index with weights frozen at the base year — isolates price change from behaviour change, same as CPI.</i>
Pt / P0 / Q0	<i>Current price / base price / base volume, per route.</i>
Volume weighting	<i>DEL-BOM with a million passengers moves the national index far more than a tiny regional route.</i>
IQR / Tukey fences	<i>Robust outlier rule for skewed data: [Q1-k-IQR, Q3+k-IQR]. Airfares are right-skewed so std-dev rules fail.</i>
k_factor	<i>Sensitivity knob, exposed not hidden, so analysts audit it; anomalies logged, never deleted.</i>

TERM DROPPED	THE LINE
Festival flag	<i>Separates transient holiday demand from structural inflation; the holidays library gives correct lunar dates.</i>
Two-tailed IQR	<i>We screen both ends — glitch-low fares like ₹1 bias the index down if you only screen the top.</i>
AI-augmented scraping	<i>Headless browsers intercept each site’s fare JSON; a vision-LLM re-derives fields when a layout changes.</i>
Self-healing	<i>Selector-based scrapers break on every UI change — ours reads the page like an analyst and re-derives the fields.</i>
Nowcasting	<i>Imputation of missing observations with confidence bands — not forecasting. How central banks nowcast GDP.</i>
SSE	<i>One-way server→browser push over plain HTTP; browser’s built-in EventSource; simpler than WebSockets for a log feed.</i>
FastAPI async	<i>The log stream and the fare fetch run concurrently in one request without blocking.</i>
Pure functions	<i>Every pipeline module takes data in, returns data out — no server, no DB — so we test and explain each alone.</i>
Postgres → SQLite failover	<i>Try Postgres, except → SQLite WAL; resilience net, not a replacement; every row tagged with its store.</i>
WAL	<i>SQLite mode where writes go to a side file first — no reader/writer blocking, crash-safe.</i>
Reproducibility	<i>Pure functions + frozen reference files + stored run record → same inputs always give the same index.</i>
Why no ML for the index	<i>This is measurement of history, not forecasting. ML adds model risk with no measurement benefit for an index.</i>
Real-time	<i>Scrapers run on a schedule and keep the cache fresh; the index always reads the latest. Hours-old is current for monthly CPI.</i>
Is scraping legal	<i>PS mandates it for a public-statistics purpose; we scrape responsibly — robots.txt, rate limits, off-peak, no resale.</i>
Desktop app	<i>pywebview wraps FastAPI in a native OS window — python run.py launches it as an application.</i>
Both DBs fail	<i>Disk is gone at that point — we’d need an in-memory retry queue; that’s stated future work, not claimed solved.</i>
Where’s the data from	<i>P0 from a Kaggle historical fares dataset, Q0 from DGCA via data.gov.in, festival dates from the holidays library.</i>
Anomaly deleted?	<i>Never — flagged, logged, stored with a reason. The index uses the clean set; the excluded rows stay on record.</i>

DRILL	The Fire Drill		REHEARSE THESE FOUR
JUDGE SAYS	YOU DO	CONSOLE PROVES	
“What if the API is down?”	Turn off wifi, click Execute	[SCRAPE][FALLBACK] ... → cached snapshot	
“What if the database dies?”	docker stop Postgres, click Execute	[DB][FALLBACK] Postgres unreachable → SQLite WAL commit OK	
“Show me k_factor is real”	Drag slider 1.5 → 3.0, re-run	[IQR] k=3.0 ... 1 excluded (was 2)	
“Prove it’s reproducible”	Run the same inputs twice	Identical Index = line + identical run id / SHA both times	

BACK PAGE	The Classifieds		GLOSSARY — EVERY TERM, ONE LINE
CORRIDOR / ROUTE — an origin–destination city pair, e.g. DEL–BOM. The unit we group fares by.	TUKEY FENCE — $Q1 - k \cdot IQR$ (low) and $Q3 + k \cdot IQR$ (high); values outside are outliers.		
BASE YEAR / BASE PERIOD — the reference the index is measured against (index = 100 there). We use 2022.	K_FACTOR — the multiplier on IQR that sets fence width; analyst-tunable, default 1.5.		
LASPEYRES INDEX — a weighted price index with quantity weights fixed at the base year.	TWO-TAILED — screening both the low and high ends, not just the high.		
PT / P0 — current / base-year average price on a route.	EX-FESTIVAL INDEX — the index recomputed with festival-window fares removed.		
Q0 — base-year passenger volume on a route (the fixed weight).	XHR INTERCEPTION — reading a page’s own background JSON responses instead of parsing its HTML.		
IQR — Interquartile Range, $Q3 - Q1$, the spread of the middle 50% of the data.	PLAYWRIGHT — a library that drives a real headless browser for automation / scraping.		

SELF-HEALING EXTRACTOR — a vision-LLM that re-derives fare fields when a site's structure changes.

ISOLATION FOREST — an unsupervised ML model that flags anomalies by how easily a point is isolated.

NOWCASTING — estimating missing current values (imputation), not predicting future ones.

SSE — Server-Sent Events; one-way server→browser streaming over HTTP; EventSource in the browser.

WEBSOCKET — a two-way persistent browser↔server channel; heavier than SSE, not needed here.

FASTAPI — an async Python web framework with built-in request validation.

UVICORN — the ASGI server process that runs a FastAPI app.

ASGI — the async server standard FastAPI is built on.

PYDANTIC MODEL — a typed schema that validates request data at the API boundary; bad input → 422.

ASYNC GENERATOR — a function that yields values over time while awaiting between them; how we stream logs.

ORCHESTRATOR — the single async generator that runs all pipeline stages and emits one log line each.

PURE FUNCTION — takes data in, returns data out, no side effects; easy to test and explain.

POSTGRESQL — the production relational database (server-based); our authoritative store.

SQLITE — a single-file database with no server; our fallback store.

WAL — Write-Ahead Logging; SQLite writes to a side file first, then merges; crash-safe, non-blocking.

DB_TARGET — the column tagging which database holds a given run row.

RUN_ID / SHA — hashes of a run's inputs; identical inputs → identical hashes → reproducible.

PYWEBVIEW — a small library that puts a web UI inside a native desktop window.

DGCA — Directorate General of Civil Aviation; publishes official Indian domestic traffic data.

MOSPI — Ministry of Statistics and Programme Implementation; owns CPI, the consumer of this index.

CPI — Consumer Price Index; the headline inflation measure this project feeds into.